

Analiza wyników pomiarowych z prób spęczania z wykorzystaniem aplikacji komputerowej

Szymon Miłkowski

12.01.2026

1 Cel i wstępne założenia

Celem tego projektu było stworzenie graficznego programu komputerowego do analizy, korekcji i transformacji danych pomiarowych z prób spęczania. Danymi wejściowymi do programu miały być pliki binarne w formacie *W01* generowane przez oprogramowanie maszyny badawczej.

Dodatkowym celem miała być kompatybilność i płynność działania zarówno na współczesnym sprzęcie, jak i starszym (Windows XP 32-bit).

2 Wybór technologii

Z racji na wymóg szerokiej kompatybilności, oraz wysokiej wydajności, konieczne było wybranie technologii pozwalającej na kompilację do natywnego kodu maszynowego z minimalną ilością zależności. Wybór padł na język C z biblioteką Raylib do obsługi grafiki, linkowaną statycznie.

Raylib posiada łatwe w obsłudze API składające się z funkcji do rysowania i obsługi wejścia (mysz, klawiatura).

3 Spęcznie

Próba spęczania polega na ściskaniu próbki wzdłuż jednej osi (pionowej) przez maszynę badawczą z zadaną prędkością. Podczas próby materiał odkształca się, zmniejszając swoją wysokość w kierunku siły oraz - zgodnie z zasadą zachowania objętości - zwiększając swoją szerokość w pozostałych kierunkach.

Podczas tego procesu, materiał wewnątrz płynie, wypychając materiał na ścianach bocznych na zewnątrz, a materiał przy podstawach trze o górną i dolną ścianę maszyny, co hamuje jego przemieszczenie na zewnątrz.

Siła nacisku (opór jaki stawia próbka) jest mierzona i zależy od materiału i fazy procesu.

3.1 Fazy procesu

Brak kontaktu

- Brak kontaktu - maszyna nie ściska próbki, zmierza w jej kierunku - stała, niska siła
- Pierwszy kontakt - luzy w maszynie są ściskane - szum informacyjny
- Faza liniowa?? - siła rośnie liniowo
- Faza nieliniowa?? - siła rośnie coraz mniej

4 Format pliku wejściowego

4.1 Nieznany format binarny

Pliki wejściowe z danymi pomiarowymi z rozszerzeniem *.W01*, zapisane są w nieudokumentowanym formacie binarnym. Wpisywanie w wyszukiwarce *.W01* lub magic number z pliku - pierwsze cztery bajty *66 66 86 3f* nie daje istotnych rezultatów. Aby zczytać dane z plików i stworzyć algorytm importujący do programu, należy więc przeprowadzić inżynierię wsteczną formatu.

4.2 Inżynieria wsteczna formatu

W inżynierii wstecznej nie ma jednego poprawnego przepisu na znalezienie rozwiązania. Każde zadanie jest osobną łamigłówką, do której musimy użyć narzędzi, doświadczenia, wiedzy informatycznej, wiedzy dziedzinowej oraz czasu. Podczas analizy od dyspozycji miałem dwa pliki: *19.v10.W01* oraz *20.v50.W01*, gdzie *v10* i *v50* odpowiadają prędkości procesu spęczania.

4.2.1 Pierwsze spojrzenie na plik

Rzut okiem w *hex-edytorze* ujawnia kilka stringów i sporo zer na początku pliku, a w dalszej części powtarzający się czterokrotnie wzór: napis i blok nieczytelnych danych.

4.2.2 Nagłówek

W nagłówku pliku, który znajduje się na początku, przed właściwymi danymi liczbowymi znajdują się metadane, informacje o rozmiarze, godzina utworzenia oraz nazwa programu wejściowego do maszyny badawczej. Cały nagłówek ma stały rozmiar 208 bajtów.

Struktura nagłówku pliku				
offset	rozmiar	wartość/typ	nazwa	komentarz
0	4	0x3f866666	magic	stała wartość określająca format
4	4	1	wersja?	najpewniej wersja formatu
8	4	$rozmiar_pliku + 1$	rozmiar	
12	4	0	nieznana	zawsze 0
16	8	"ScriptPl"	stały napis	8 znaków, bez null terminatora
24	4	$rozmiar_pliku - 16$	rozmiar2	wygląda jak zbędna informacja, może oznaczać "rozmiar począwszy od napisu"
28	4	liczba wierszy	liczba wierszy	ilość liczb w każdej kolumnie danych
32	4	liczba kolumn	liczba kolumn	ilość kolumn z danymi
36	4	1	nieznane	
40	4	1	nieznane	
44	18	"dd-MM-yyyyhh:mm:ss"	czas utworzenia	napis bez null terminatora
62	42	nieznana	nieznane	42 bajty, głównie zera
104	104	nazwa pliku	plik badania	nazwa pliku wejściowego do maszyny, dopełniona zerami do 104 bajtów

4.2.3 Kolumny (serie danych)

Kolumny mają zmienny rozmiar, zależnie od pliku, jednak zawsze taki sam w obrębie jednego pliku.

Struktura kolumny				
offset	rozmiar	wartość/typ	nazwa	komentarz
0	4	int	nieznane	1 lub 16385
4	59	napis	nazwa	nazwa i jednostka kolumny, oddzielone spacjami
63	$4 * liczba_wierszy$	float	dane	tablica 32-bitowych liczb zmiennoprzecinkowych

4.2.4 Struktura całego pliku

Plik składa się z nagłówka o stałym rozmiarze i zmiennej liczby kolumn zmiennej długości, które zawierają swój nagłówek oraz danych liczbowych.

Struktura pliku	
rozmiar	element
208	nagłówek
$63 + 4 * liczba_wierszy$	kolumna 1
$63 + 4 * liczba_wierszy$	kolumna 2
...	...
$63 + 4 * liczba_wierszy$	kolumna n

5 Ogólna struktura aplikacji

5.1 Komponenty

Interfejs graficzny aplikacji podzielony jest na "komponenty" - funkcje zawierające logikę kontrolną oraz rysujące pewien element interfejsu. Wywołanie tej funkcji oznacza, że komponent istnieje w obecnej klatce. Funkcje te mogą wywoływać inne funkcje komponentów, co tworzy swego rodzaju strukturę drzewa komponentów, rozpoczynającą się w głównej pętli programu. Pozycja w strukturze drzewa nie ma jednak wpływu na rozmieszczenie elementu, pozycja jest statyczna, lub określana poprzez podane parametry.

5.2 Zarządzanie stanem

Inspirowane działaniem innego drzewiastego systemu komponentów graficznych - webowym Reactem - komponenty, jeśli to konieczne, operują na dwóch rodzajach danych: właściwości (*props*) i stan (*state*).

Właściwości są podawane do funkcji komponentu jako kopia (pass by value), bądź stały wskaźnik, mają za zadanie sterować zachowaniem i wyglądem komponentu.

Stan jest podawany jako wskaźnik do pamięci zarządzanej przez rodzica (komponent wywołujący), ma za zadanie przechowywanie stanu komponentu między wywołaniami, jest modyfikowany głównie przez sam komponent oraz jego dzieci (jeśli zostanie podany do nich jako ich stan).

Stan globalny jest wykorzystywany w niektórych komponentach, które z założenia mają istnieć tylko w jednej sztuce jednocześnie, jak lista otwartych plików lub narzędzia diagnostyczne *FPSCounter* i *Clock*, jako zmienne globalne w jednostce translacyjnej (niewidoczno poza obecnym plikiem .c), lub statyczne zmienne lokalne w funkcji komponentu.

6 Rysowanie wykresu

Raylib zawiera jedynie proste funkcje rysowania kształtów. Jedną z nich jest *DrawSplineLinear*, łącząca podane współrzędne pikseli liniowymi odcinkami. cośtam właśnie jej użyjemy bla

```
DrawSplineLinear(state->point_buffer, state->visible.count, 2.f, DARKBLUE);
```

6.1 Poruszanie się w widoku

Aby zwiększyć interaktywność, użytkownik może się poruszać po płaszczyźnie XY, przeciągając płaszczyzną myszką, oraz przybliżając i oddalając kółkiem myszy. Manipulowane są w ten sposób zmienne skali oraz przesunięcia uwzględnione w przekształceniu punktów ze współrzędnych kartezjańskich na współrzędne ekranu.

6.2 Transformacja współrzędnych

6.2.1 Model matematyczny

Punkt podlega przekształceniu liniowemu:

$$x = \frac{z(x_p - x_v) - x_S}{x_E - x_S} * W + x_O \quad (1)$$

gdzie

z - współczynnik skali

x_p - współrzędna x punktu w układzie kartezjańskim

x_v - przesunięcie x w układzie kartezjańskim

x_S - współrzędna x lewej krawędzi układu kartezjańskiego przy zerowym przesunięciu

x_E - współrzędna x prawej krawędzi układu kartezjańskiego przy zerowym przesunięciu

W - szerokość wykresu w pikselach

x_O - odległość prawej krawędzi wykresu od prawej krawędzi ekranu

$$y = \frac{-z(y_p + y_v) - y_S}{y_E - y_S} * H + y_O \quad (2)$$

gdzie

z - współczynnik skali

y_p - współrzędna y punktu w układzie kartezjańskim

y_v - przesunięcie y w układzie kartezjańskim

y_S - współrzędna y dolnej krawędzi układu kartezjańskiego przy zerowym przesunięciu

y_E - współrzędna y górnej krawędzi układu kartezjańskiego przy zerowym przesunięciu

H - wysokość wykresu w pikselach

y_O - odległość górnej krawędzi wykresu od górnej krawędzi ekranu

Zmiana znaku y_p jest spowodowana, tym że oś $+Y$ we współrzędnych ekranowych skierowana jest w dół.

6.2.2 Implementacja

W kodzie programu zaimplementowane jest to dwuetapowo.

Obliczanie granic widocznego fragmentu

```
Bounds compute_bounds(const float zoom, const Vector2 pan) {
    return (Bounds) {
        .start_x = PLOT_START_X / zoom + pan.x,
        .end_x = PLOT_END_X / zoom + pan.x,
        .start_y = PLOT_START_Y / zoom / ((float) PLOT_WIDTH / PLOT_HEIGHT) + pan.y,
        .end_y = PLOT_END_Y / zoom / ((float) PLOT_WIDTH / PLOT_HEIGHT) + pan.y,
    };
}
```

Transformacja pojedynczego punktu

```
Vector2 transform_v_to_pixel(const Vector2 v, const Bounds bounds) {
    return (Vector2){
        .x = (v.x - bounds.start_x) / (bounds.end_x - bounds.start_x) * PLOT_WIDTH + PLOT_OFFSET_X,
        .y = (-v.y - bounds.start_y) / (bounds.end_y - bounds.start_y) * PLOT_HEIGHT + PLOT_OFFSET_Y,
    };
}
```

Transformacja wielu punktów jednocześnie jest zoptymalizowana pod kątem ilości niepotrzebnych operacji (opisana w punkcie 6.3.3).

6.3 Minimalizacja renderowanych punktów

6.3.1 Opis problemu

Podczas przeglądania wykresu, nie każdy punkt będzie widoczny jednocześnie. Chcąc zminimalizować ilość punktów podlegających transformacji oraz rysowaniu, należy znaleźć zakres punktów widocznych na ekranie.

6.3.2 Szukanie pierwszego punktu

Zakładając, że współrzędna x w zestawie danych jest niemalejąca, możemy potraktować zbiór X jako posortowany zbiór danych i użyć wydajnego algorytmu **wyszukiwania binarnego**, o złożoności obliczeniowej $O(\log_2 n)$.

```

int find_first_visible(const Vector2* data, const int count, const float start_x) {
    if (data[count - 1].x < start_x) return -1;

    int a = 0, b = count - 1;
    while (b - a > 1) {
        const int mid = (a + b) / 2;
        if (data[mid].x > start_x) b = mid;
        else a = mid;
    }
    return a;
}

```

6.3.3 Transformacja punktów

Po znalezieniu pierwszego elementu zbioru wyjściowego, współrzędne są przekształcane i zapisywane w tablicy wyjściowej aż do napotkania punktu znajdującego się za prawą krawędzią widoku lub końca tablicy wejściowej. Wyszukiwanie binarne nie ma tutaj zastosowania, ponieważ i tak każdy widoczny punkt musi zostać przekształcony, co wiąże się z liniowym algorytmem.

```

VisiblePointsInfo compute_visible_points_offset(
    const DataSource data_source, Vector2* point_buffer, const Bounds bounds, const Vector2 offset
) {
    VisiblePointsInfo res = {
        .start = find_first_visible(data_source.data, data_source.count, bounds.start_x - offset.x)
    };
    if (res.start == -1) return res;

    const float x_factor = 1 / (bounds.end_x - bounds.start_x) * PLOT_WIDTH;
    const float y_factor = -1 / (bounds.end_y - bounds.start_y) * PLOT_HEIGHT;

    const int visible_count = data_source.count - res.start;
    for (int i = 0; i < visible_count; i++) {
        const int i_source = i + res.start;
        point_buffer[i].x = (data_source.data[i_source].x - bounds.start_x + offset.x)
            * x_factor + PLOT_OFFSET_X;
        point_buffer[i].y = (data_source.data[i_source].y + bounds.start_y - offset.y)
            * y_factor + PLOT_OFFSET_Y;
        if (data_source.data[i_source].x > bounds.end_x - offset.x) {
            res.count = i + 1;
            return res;
        }
    }
    res.count = visible_count;
    return res;
}

```

Algorytm nie używa funkcji do transformacji pojedynczego punktu, co pozwala na obliczenie niezmiennych współczynników poza pętlą, unikając zbędnych operacji.

6.4 Wykrywanie zmian

Transformacja wszystkich widocznych punktów jest mimo wszystko kosztowną operacją, dlatego wykonywana jest jedynie gdy reprezentacja punktów na wykresie powinna ulec zmianie. Odpowiada za to zmienna *change*, zawierająca flagi sterujące rekalkulacją elementów wykresu, ustawiana przez funkcję odpowiadającą za obsługę wejścia.

7 Analiza danych spęczania

7.1 Charakterystyka danych

Dane zawarte w plikach z pomiaru próby spęczania zawierają 4 kolumny

7.1.1 Load[kN]

Siła zmierzona przez maszynę - opór, który stawia próbka. Wartości ujemne.

7.1.2 Stroke[mm]

Przemieszczenie narzędzia maszyny. Wartości ujemne.

7.1.3 Command[%]

7.1.4 Time[s]

Czas obecnego rekordu, wartość rosnąca od 0, co 0.0025.

7.2 Fazy procesu w danych

- **Brak kontaktu** - obciążenie jest stałe lub z minimalnymi wahaniami, niewielkie
- **Pierwszy kontakt** - obciążenie waha się nierównomiernie, zauważalnie bardziej niż w poprzednim etapie
- **Liniowa** - obciążenie rośnie liniowo
- **Nieliniowa** - wzrost obciążenia stopniowo i płynnie maleje

7.3 Wykrywanie faz

Fazy procesu są na tyle charakterystyczne, że można je wykryć z poziomu algorytmu do napisania jeszcze

7.4 Analiza fazy liniowej

7.5 Analiza fazy nieliniowej

8 Proces optymalizacji algorytmu regresji wielomianowej

8.1 Wstęp

Ten rozdział opisuje proces optymalizacji algorytmu wykonującego regresję wielomianową podanego stopnia dla podanych punktów. Udokumentowane są użyte techniki oraz pomiary czasowe. Przed optymalizacją, algorytm ten był zdecydowanie najbardziej czasochłonną operacją w całym programie.

8.2 Dane wejściowe, sprzęt i środowisko

Podczas testowania przyjęto następujące parametry:

- **Punkty wejściowe:** 29 punktów na krzywej $2\sin x + 2$, dla x od -5, co 0.3
- **Stopień szukanego wielomianu:** 28
- **CPU:** AMD Zen2 Ryzen 3600
- **Kompilator:** GCC 15.2.1
- **Opcje kompilatora w trybie Debug:** -g
- **Opcje kompilatora w trybie Release:** -O3

8.3 Sposób pomiarów

8.3.1 Realny benchmark

Pomiary algorytmu oraz jego fragmentów miały miejsce jako mierzenie części całego działającego programu, maksymalnie raz na klatkę, przy zachowaniu całej jego pozostałej funkcjonalności. Rezultat był wykorzystany do wyświetlenia na żywo wykresu wyznaczonej funkcji, nie ma mowy więc o pomijaniu przez kompilator operacji, których wynik nie jest używany. Dodatkowo w syntetycznym benchmarku, wywołującym w pętli funkcję wiele razy, korzystniej mogłaby być wypełniona pamięć cache oraz bardziej dokładny predyktor skoków, dlatego wplecenie pomiarów w standardowy przepływ programu lepiej oddaje wydajność w realnych warunkach.

8.3.2 RDTSC

Architektura x86 implementuje instrukcję **RDTSC** (*Read Time-Stamp Counter*), zwracającą obecną wartość licznika cykli. Licznik służy do precyzyjnego pomiaru czasu i jest inkrementowany co stałą jednostkę czasu, odpowiadającą częstotliwości procesora. Dynamiczne taktowanie obecne we współczesnych rdzeniach nie ma wpływu na częstotliwość tego licznika. W przypadku sprzętu pomiarowego jest to 3.6GHz.

8.3.3 Implementacja pomiaru

W momencie uruchomienia benchmarku ustawiana jest flaga (pkt. 6.4), zapewniająca że komponent wielomianu otrzyma zmianę w każdej klatce, co wywoła pożądaną funkcję. Flaga będzie ustawiana, dopóki benchmark nie zbierze 10000 pomiarów.

Rozpoczęcie pomiaru

```
void clock_start() {  
    #ifndef __arm64  
        start_cycles = __rdtsc();  
    #endif  
    start = clock();  
}
```

Funkcja ustawia zmienne globalne licznika cykli używając funkcji wewnętrznej `__rdtsc` oraz drugiego licznika używając kanonicznego `clock`. Użycie `__rdtsc` jest wyłączone na platformie ARM, gdzie instrukcja RDTSC nie jest dostępna.

Zakończenie pomiaru

```
void clock_end() {
#ifdef __arm64
    end_cycles = __rdtsc();
#endif
    end = clock();
    const clock_t diff = end - start;
    const unsigned long long diff_cycles = end_cycles - start_cycles;

    if (samples == -1) return;
    if (samples < CLOCK_SAMPLE_COUNT) {
        samples++;
        sum += diff;
        sum_cycles += diff_cycles;
        if (best == -1 || diff < best) best = diff;
        if (best_cycles == -1 || diff_cycles < best_cycles) best_cycles = diff_cycles;

        if (samples == CLOCK_SAMPLE_COUNT) {
            /* wyświetlanie wyniku... */
            samples = -1;
            sum = 0;
            sum_cycles = 0;
            best = -1;
            best_cycles = -1;
        }
    }
}
```

Liczona jest różnica w z obecnym a poprzednim stanem licznika. Wartość -1 dla `samples` oznacza wyłączony benchmark, a dla `best` i `best_cycles` - brak wartości.

8.3.4 Narzut pomiaru

Aby móc wyciągnąć wnioski z pomiarów sekcji kodu, należy najpierw zmierzyć czas wykonania samego kodu pomiarowego. Wszelkie wyniki zbliżone do narzutu powiaru są niemiernodajne.

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	3642	2592	0.000814	0.000000
2	10000	3561	2484	0.000791	0.000000
3	10000	3604	2520	0.000814	0.000000
5	10000	3683	2592	0.000810	0.000000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	3880	2628	0.000817	0.000000
2	10000	3615	2556	0.000789	0.000000
3	10000	3713	2736	0.000796	0.000000
4	10000	3569	2700	0.000775	0.000000

Czas jest na tyle mały, że `clock` wskazuje 0 - jest za mało precyzyjny. Co ciekawe, wygląda na to, sam narzut w konfiguracji release jest nieco wolniejszy niż w debug.

8.4 Pierwotny algorytm

Poniższy kod jest pierwszą poprawną wersją naiwnej implementacji algorytmu regresji. W jego skład wchodzi przygotowanie pamięci, wypełnienie macierzy i wektora sumami, rozwiązanie układu równań eliminacją Gaussa oraz przepisanie i zwrócenie wyniku.

8.4.1 Całość

Kod

```
// makra dla przejrzystości
#define _get_mat_cell(row, col) matrix[swap_table[row] * m + col]
#define _get_error(row) errors[swap_table[row]]
#define _get_coeff(row) coeffs[swap_table[row]]

void regression(CurvePolynomial* curve, const Vector2* points, const int point_count) {
    clock_start();
    // przygotowanie
    const int degree = point_count - 1;
    const int n = point_count;
    const int m = degree;

    float* matrix = malloc((m*m + 2*m) * sizeof (float) + m * sizeof (int));
    float* errors = &matrix[m*m];
    float* coeffs = &errors[m];
    int* swap_table = (int*) &coeffs[m];

    // wypełnianie macierzy i wektora
    for (int row = 0; row < m; ++row) {
        swap_table[row] = row;

        for (int col = 0; col < m; ++col) {
            float cell = 0;
            const int power = row + col;
            for (int i = 0; i < n; ++i) {
                cell += pownf(points[i].x, power);
            }
            _get_mat_cell(row, col) = cell;
        }
        float error = 0;
        for (int i = 0; i < n; ++i) {
            error += pownf(points[i].x, row) * points[i].y;
        }
        _get_error(row) = error;
    }

    // eliminacja Gaussa - postępowanie proste
    for (int i = 0; i < m - 1; ++i) {
        float max = fabsf(_get_mat_cell(i, i));
        int max_row = i;
        for (int row = i + 1; row < m; ++row) {
            const float current_abs = fabsf(_get_mat_cell(row, i));
            if (current_abs > max) {
                max = current_abs;
                max_row = row;
            }
        }
        if (max_row != i) swap_rows(swap_table, i, max_row);

        const float cell = _get_mat_cell(i, i);
```

```

for (int row = i + 1; row < m; ++row) {
    const float coeff = _get_mat_cell(row, i) / cell;
    _get_error(row) -= coeff * _get_error(i);
    for (int col = 0; col < m; ++col) {
        _get_mat_cell(row, col) -= coeff * _get_mat_cell(i, col);
    }
}
}

// eliminacja Gaussa - postępowanie odwrotne
_get_coeff(m - 1) = _get_error(m - 1) / _get_mat_cell(m - 1, m - 1);
for (int i = m - 2; i >= 0; --i) {
    float sigma = 0;
    for (int k = i + 1; k < m; ++k) {
        sigma += _get_mat_cell(i, k) * _get_coeff(k);
    }
    _get_coeff(i) = (_get_error(i) - sigma) / _get_mat_cell(i, i);
}

// przepisanie wyników i zwolnienie pamięci
for (int i = 0; i < m; ++i) {
    curve->coefficients[i] = _get_coeff(i);
}
free(matrix);

curve->order = m;
curve->start_x = points[0].x;
curve->end_x = points[point_count - 1].x;
clock_end();
}

```

Benchmark

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	1228592	1182240	0.340353	0.328000
2	10000	1222491	1187676	0.338560	0.329000
3	10000	1221949	1184508	0.338329	0.328000
4	10000	1219704	1187136	0.337932	0.328000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	951561	913932	0.263545	0.253000
2	10000	952509	913644	0.263831	0.253000
3	10000	939618	913608	0.260251	0.252000
4	10000	942651	914544	0.261089	0.253000

Komentarz Wykonanie mieści się poniżej 1 milisekundy, jednak zajmuje dużą ilość czasu w porównaniu do reszty programu oraz powoduje zauważalne spadki liczby klatek na sekundę (z ok. 8000 do ok. 1000).

8.4.2 Fragment 1 - Alokacja i przygotowanie pamięci

Alokacja pamięci na stercie jest uważana za powolną operację, więc mimo że wyoknujemy ją tylko raz, należy dla pewności ją zmierzyć. Alokacja jest już wstępnie zoptymalizowana, zamiast czterech osobnych alokacji dla czterech tablic, mamy jeden blok pamięci zdolny zmieścić wszystkie tablice, których wskaźniki ustawiamy z odpowiednim przesunięciem.

Kod

```
clock_start();
const int degree = point_count - 1;
const int n = point_count;
const int m = degree;

float* matrix = malloc((m*m + 2*m) * sizeof (float) + m * sizeof (int));
float* errors = &matrix[m*m];
float* coeffs = &errors[m];
int* swap_table = (int*) &coeffs[m];
clock_end();
```

Benchmark

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	4331	2952	0.000950	0.000000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	4131	3168	0.000930	0.000000

8.4.3 Fragment 2 - Wypełnianie macierzy i wektora błędów

Kod

```
clock_start();
for (int row = 0; row < m; ++row) {
    swap_table[row] = row;

    for (int col = 0; col < m; ++col) {
        float cell = 0;
        const int power = row + col;
        for (int i = 0; i < n; ++i) {
            cell += pownf(points[i].x, power);
        }
        _get_mat_cell(row, col) = cell;
    }
    float error = 0;
    for (int i = 0; i < n; ++i) {
        error += pownf(points[i].x, row) * points[i].y;
    }
    _get_error(row) = error;
}
clock_end();
```

Benchmark

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	1064016	1010808	0.294737	0.281000
2	10000	1045250	1008720	0.289477	0.280000
3	10000	1049505	1009584	0.290731	0.280000
4	10000	1056144	1012932	0.292435	0.281000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	938385	898308	0.259947	0.248000
2	10000	927664	898488	0.256944	0.248000
3	10000	922574	898020	0.255469	0.248000
4	10000	930611	898092	0.257770	0.248000

8.4.4 Fragment 3 - Rozwiązanie układu równań

Kod

```
clock_start();
for (int i = 0; i < m - 1; ++i) {
    float max = fabsf(_get_mat_cell(i, i));
    int max_row = i;
    for (int row = i + 1; row < m; ++row) {
        const float current_abs = fabsf(_get_mat_cell(row, i));
        if (current_abs > max) {
            max = current_abs;
            max_row = row;
        }
    }
    if (max_row != i) swap_rows(swap_table, i, max_row);

    const float cell = _get_mat_cell(i, i);
    for (int row = i + 1; row < m; ++row) {
        const float coeff = _get_mat_cell(row, i) / cell;
        _get_error(row) -= coeff * _get_error(i);
        for (int col = 0; col < m; ++col) {
            _get_mat_cell(row, col) -= coeff * _get_mat_cell(i, col);
        }
    }
}

_get_coeff(m - 1) = _get_error(m - 1) / _get_mat_cell(m - 1, m - 1);
for (int i = m - 2; i >= 0; --i) {
    float sigma = 0;
    for (int k = i + 1; k < m; ++k) {
        sigma += _get_mat_cell(i, k) * _get_coeff(k);
    }
    _get_coeff(i) = (_get_error(i) - sigma) / _get_mat_cell(i, i);
}
clock_end();
```

Benchmark

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	187736	180720	0.051796	0.047000
2	10000	184986	180684	0.051078	0.048000
3	10000	185314	180900	0.051125	0.048000
4	10000	186258	180792	0.051409	0.049000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	13700	12240	0.003580	0.000000
2	10000	13416	12204	0.003531	0.003000
3	10000	13402	12348	0.003536	0.001000
4	10000	13398	12240	0.003525	0.000000

8.4.5 Fragment 4 - Przepisywanie wyniku i zwalnianie pamięci

Wynikowa tablica współczynników musi zostać przepisana, ponieważ zamiana wierszy w eliminacji Gaussa zmienia ich kolejność w pamięci.

Kod

```
clock_start();
for (int i = 0; i < m; ++i) {
    curve->coefficients[i] = _get_coeff(i);
}
free(matrix);

curve->order = m;
curve->start_x = points[0].x;
curve->end_x = points[point_count - 1].x;
clock_end();
```

Benchmark

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	3960	3132	0.000895	0.000000
2	10000	3953	3132	0.000887	0.000000
3	10000	3925	3132	0.000885	0.000000
4	10000	4049	3168	0.000900	0.000000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	3809	3168	0.000859	0.000000
2	10000	3921	2844	0.000870	0.000000
3	10000	3844	3096	0.000854	0.000000
4	10000	3712	2880	0.000842	0.000000

8.4.6 Obserwacje i wnioski

Czas zarządzania pamięcią i przepisywania wyniku jest pomijalny. Zdecydowana większość czasu jest skoncentrowana we fragmencie nr 2 - wypełnianiu macierzy i wektora. To na nim należy się skupić pod kątem optymalizacji. Rozwiązywanie układu zajmuje zauważalną, lecz dużo mniejszą ilość czasu.

8.5 Zmiana funkcji potęgi

8.5.1 Idea

Pierwotna wersja używa funkcji `pownf` ze standardowej biblioteki C. Funkcja ta przyjmuje jako wykładnik 64-bitową liczbę całkowitą ze znakiem. Ideą za tą zmianą była szansa poprawy wydajności poprzez zignorowanie przypadku dla liczb ujemnych, ponieważ w naszym algorytmie nie występują ujemne wykładniki, oraz zmniejszenie rozmiaru wykładnika, co pozwoli uniknąć konwersji z liczby 32-bitowej na 64-bitową oraz zaoszczędzi miejsce w pamięci cache.

8.5.2 Kod

Oba wywołania `pownf` zostały zastąpione wywołaniem własnej funkcji potęgi. Jest to prosty algorytm iteracyjny.

```
float power_fi(const float x, int pow) {
    if (pow == 0) return 1;
    float res = x;
    while (--pow) {
        res *= x;
    }
    return res;
}
```

8.5.3 Benchmark

Pomiar obejmuje cały fragment 2 ze zmienioną funkcją potęgi.

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	4380990	4185072	1.214246	1.163000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	671631	646632	0.185943	0.179000
2	10000	672397	644472	0.186236	0.175000
3	10000	675776	648216	0.187136	0.179000
4	10000	670312	647784	0.185646	0.179000

8.5.4 Obserwacje i wnioski

W trybie debug zmiana spowodowała ponad 4-krotny spadek wydajności, a w trybie release - ok. 28% wzrost. Oznacza to, że początkowe założenia były nietrafione i nowy algorytm jest bardzo wolny, jednak kompilator z włączoną optymalizacją zauważył co próbowaliśmy osiągnąć i podmienił na wydajniejszą implementację.

8.6 Usuwanie redundancji

8.6.1 Idea

Wracając do wzoru elementów macierzy, można zauważyć, że jest ona bardzo uporządkowana. Co więcej, większość sum pojawia się w niej więcej niż jeden raz. Jeśli weźmiemy elementy od $a_{1,1}$ do $a_{m,m}$, wartość elementu $a_{2,1}$ pojawi się 2-krotnie, wartość $a_{3,1}$ - 3-krotnie, a wartość $a_{m,1}$ - m -krotnie. Aby uniknąć zbędnych obliczeń tej samej wartości, tworzymy tablicę sum od $\sum_{i=1}^n x_i^0$ do $\sum_{i=1}^n x_i^{2m}$, z której będziemy kopiować wartości w odpowiednie elementy macierzy.

8.6.2 Kod

```
clock_start();
// wypełnianie tablicy sum
for (int pow = 0; pow < 2*m; ++pow) {
    float total = 0;
    for (int i = 0; i < n; ++i) {
        total += pownf(points[i].x, pow);
    }
    powers[pow] = total;
}

for (int row = 0; row < m; ++row) {
    swap_table[row] = row;

    // wypełnianie macierzy
    for (int col = 0; col < m; ++col) {
        matrix[row * m + col] = powers[row + col];
    }

    /* wypełnianie wektora
    ...
    */
}
clock_end();
```

8.6.3 Benchmark - z pownf

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	125819	120420	0.034684	0.028000
2	10000	124391	120348	0.034308	0.031000
3	10000	124605	120420	0.034354	0.033000
4	10000	127579	120600	0.035173	0.032000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	102549	96264	0.028172	0.026000
2	10000	103316	96156	0.028430	0.025000
3	10000	98939	95940	0.027211	0.025000
4	10000	101698	96120	0.027971	0.025000

8.6.4 Benchmark - z własną funkcją potęgi

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	420020	406764	0.116187	0.111000
2	10000	422607	408204	0.116971	0.112000
3	10000	420295	408312	0.116333	0.102000
4	10000	419029	408492	0.115991	0.113000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	68148	65268	0.018696	0.017000
2	10000	67473	65268	0.018504	0.016000
3	10000	67534	65124	0.018524	0.016000
4	10000	68253	65412	0.018719	0.016000

8.6.5 Obserwacje i wnioski

Zapisanie wyniku sum dało bardzo znaczące przyspieszenie. W każdej konfiguracji fragment wykonuje się około 10-krotnie szybciej.

8.7 Wypełnianie macierzy i wektora - pomiar fragmentów

8.7.1 Fragment 1 - Obliczanie sum

Kod

```
clock_start();
for (int pow = 0; pow < 2*m; ++pow) {
    float total = 0;
    for (int i = 0; i < n; ++i) {
        total += power_fi(points[i].x, pow);
    }
    powers[pow] = total;
}
clock_end();
```

Benchmark

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	337407	327492	0.093372	0.090000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	61379	57600	0.016807	0.014000

8.7.2 Fragment 2 - Kopiowanie sum do macierzy

Kod

```
clock_start();
for (int row = 0; row < m; ++row) {
    swap_table[row] = row;

    for (int col = 0; col < m; ++col) {
        matrix[row * m + col] = sum_of_powers[row + col];
    }
}
clock_end();
```

Benchmark

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	11213	9216	0.002846	0.000000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	3972	3132	0.000914	0.000000

8.7.3 Fragment 3 - Wypełnianie wektora

Kod

```
clock_start();
for (int row = 0; row < m; ++row) {
    float error = 0;
    for (int i = 0; i < n; ++i) {
        error += power_fi(points[i].x, row) * points[i].y;
    }
    errors[row] = error;
}
clock_end();
```

Benchmark

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	77407	75024	0.021263	0.019000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	15495	13500	0.004107	0.002000

8.7.4 Obserwacje i wnioski

Najwięcej czasu zajmuje obliczanie sum potęg. Kilkukrotnie mniej - wypełnianie wektora, a w trybie release wypełnianie macierzy jest pomijalne.

8.8 Eliminacja potęgowania - podejście iteracyjne

8.8.1 Idea

Potęgowanie poprzez wielokrotne mnożenie wymaga dużej ilości operacji mnożenia. Dodatkowo, są to operacje liniowo zależne od siebie, więc procesor musi je wykonać w prawidłowej kolejności jedna po drugiej. Niweluje to sposoby przyspieszania obliczeń na współczesnym sprzęcie, takie jak OoO (Out of Order execution) oraz SIMD (Single Instruction, Multiple Data). Złożoność obliczeniowa takiego algorytmu to $O(n) = n$, gdzie n to wykładnik.

Algorytm wypełniający tablicę sum wykorzystuje $2mn$ potęg, zatem zakładając, że $n = m - 1$, ilość operacji mnożenia jest równa:

$$O(m) = 2m^2n = 2(m^3 - m^2) \quad (3)$$

Rozwiązaniem tego problemu jest zmiana podejścia z obliczania każdej potęgi osobno:

$$x^n = x * x * \dots * x \quad (4)$$

na obliczanie następnej potęgi na podstawie poprzedniej:

$$x^n = x^{n-1} * x \quad (5)$$

Dzięki temu każda następna potęga wymaga jedynie jednej operacji mnożenia i ich ilość w całym algorytmie zmienia się na:

$$O(m) = 2(m^2 - m) \quad (6)$$

8.8.2 Kod

```
clock_start();
powers[0] = (float) n;
powers[1] = 0;
for (int i = 0; i < n; ++i) {
    const float x = points[i].x;
    points_x[i] = x;
    points_x_powers[i] = x;
    powers[1] += x;
}
for (int pow = 2; pow < 2 * m; ++pow) {
    float total = 0;
    for (int i = 0; i < n; ++i) {
        points_x_powers[i] *= points_x[i];
        total += points_x_powers[i];
    }
    powers[pow] = total;
}
clock_end();
```

8.8.3 Benchmark

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	23006	21384	0.006192	0.004000
2	10000	23342	21276	0.006276	0.004000
3	10000	23301	21456	0.006283	0.004000
4	10000	23267	21312	0.006265	0.004000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	6546	5364	0.001625	0.000000
2	10000	6418	5436	0.001594	0.000000
3	10000	6423	5220	0.001588	0.000000
4	10000	6315	5364	0.001559	0.000000

8.8.4 Wypełnianie wektora - to samo podejście

Tożsame usprawnienie można zastosować w drugim fragmencie stosującym potęgowanie - wypełnianiu wektora. Tablicę pomocniczą wypełniamy wartościami y , a w każdej następnej iteracji mnożymy przez odpowiednie wartości x .

8.8.5 Wypełnianie wektora - Kod

```
clock_start();
errors[0] = 0;
for (int i = 0; i < n; ++i) {
    const float y = points[i].y;
    points_x_powers_y[i] = y;
    errors[0] += y;
}
for (int pow = 1; pow < m; ++pow) {
    float total = 0;
    for (int i = 0; i < n; ++i) {
        points_x_powers_y[i] *= points_x[i];
        total += points_x_powers_y[i];
    }
    errors[pow] = total;
}
clock_end();
```

8.8.6 Wypełnianie wektora - Benchmark

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	13730	12312	0.003610	0.002000
2	10000	14205	12636	0.003702	0.001000
3	10000	13688	12636	0.003587	0.000000
4	10000	13778	12492	0.003618	0.001000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	4839	3888	0.001150	0.000000
2	10000	4755	4032	0.001135	0.000000
3	10000	4782	3744	0.001139	0.000000
4	10000	4807	3924	0.001144	0.000000

8.8.7 Obserwacje i wnioski

Zmiana spowodowała ogromny wzrost wydajności, ponad 10-krotny dla tablicy potęg oraz ponad 3-krotny dla wektora. Przyczynia się od tego redukcja złożoności obliczeniowej oraz rozłożenie danych. Wykonywanie tej samej operacji na wielu niezależnych elementach tablicy bardzo sprzyja wykonywaniu poza kolejnością oraz operacji SIMD. Własna funkcja potęgowanie też nie jest już używana, więc można ją usunąć.

8.9 Podsumowanie

8.9.1 Cały algorytm przed optymalizacją

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	1228592	1182240	0.340353	0.328000
2	10000	1222491	1187676	0.338560	0.329000
3	10000	1221949	1184508	0.338329	0.328000
4	10000	1219704	1187136	0.337932	0.328000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	951561	913932	0.263545	0.253000
2	10000	952509	913644	0.263831	0.253000
3	10000	939618	913608	0.260251	0.252000
4	10000	942651	914544	0.261089	0.253000

8.9.2 Cały algorytm przed optymalizacją

Debug					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	227112	219996	0.062745	0.060000
2	10000	226345	219816	0.062483	0.059000
3	10000	226825	220788	0.062675	0.061000
4	10000	226253	220104	0.062492	0.060000

Release					
Pomiar	Liczba próbek	Średnia RDTSC	Min RDTSC	Średnia clock [ms]	Min clock [ms]
1	10000	20354	17460	0.005447	0.003000
2	10000	19150	17244	0.005123	0.003000
3	10000	19586	17424	0.005245	0.003000
4	10000	19994	17568	0.005364	0.002000

8.9.3 Wnioski

Optymalizacja okazała się bardzo skuteczna, stosując różne techniki udało się poprawić wydajność algorytmu **45-50-krotnie**.